Project Plan

Agile Software Engineering Capstone

Document field	Team entry
Project title	'Royal Budget' – A Personal Finance & Budgeting Application
Team name	Royal Development
Team members	Jake Hall, Cesar Giner, Jacob Melookaran, Joshua Webber
Student document owner	Jacob Melookaran
Instructor	Shah, Syed Jawad Hussain
Sponsor or client	Commerce Bank
Version and date	1.0, November 7, 2026
Approval status	In review

Revision history

Version	Date	Author	Summary of change	Reviewer or approval
1.0	November 7, 2026	Cesar Giner	First full plan, written during Sprint 4. Records Sprints 0–3 plans Sprints 4–7, including the changes from the midterm review: post-demo fixes, the Profile and Settings page, tests, the AI assistant, and the two stretch goals.	Team review

Document outline

Section	Purpose
1 Plan overview	Project goal, MVP, planning approach, priorities, and constraints
2 Release strategy	Major deliverables, milestones, releases, and sprint roadmap
3 Backlog and work breakdown	Prioritized work, WBS, and work package details
4 Estimation and capacity	Effort, duration, assumptions, and available team time
5 Schedule and dependencies	Task relationships, dates, critical work, and resource balancing
6 Sprint operating plan	Sprint cadence, iteration plan, Definition of Ready, and Definition of Done
7 Roles and responsibilities	Ownership, backup coverage, and decision authority
8 Communication plan	Information needs, channels, timing, and escalation
9 Quality plan	Reviews, testing, acceptance, defects, and documentation

Section	Purpose
10 Risks and issues	Possible problems, active problems, responses, and owners
11 Change and configuration control	How the team manages scope, code, documents, and releases
12 Progress tracking	Measures, reports, forecasts, and plan updates
13 Tools and environments	Shared tools, access, and development or test environments
14 Review and submission	Approvals and final completeness check

1 Plan overview

This section gives the reader a quick picture of what the team plans to deliver and how the team will organize the work.

1.1 Product goal and minimum viable product

Royal Budget helps people build better budgeting habits. A user creates a monthly budget with a plan (Balanced 50/30/20, Save more 40/20/40, or Essentials 60/20/20), records fabricated income and expenses, and compares planned spending with actual spending.

The minimum viable product is the sponsor-required core: registration, sign in and secure sessions, the dashboard, monthly budgets and budget progress, transactions, the Profile and Settings page, and unit tests. The AI budget assistant and the two chosen stretch goals (Data Visualization and an OWASP ZAP scan) are committed next. Deployment, notifications, and other features are optional and are cut first if time runs short.

1.2 Planning approach and planning horizon

The team uses a mixed approach. Fixed dates from the charter and course (midterm demo, feature complete, and final delivery) are planned up front; the work inside each sprint is planned adaptively at sprint planning and adjusted after each review.

Sprints are two weeks long, Friday to Thursday, from Sprint 0 (September 4) to Sprint 7 (December 15). Sprint 6 is shortened to end at feature complete on December 7, and Sprint 7 is a one-week release sprint. The current sprint is planned in detail as tasks; the next sprint as estimated stories; later sprints only as goals.

1.3 Priorities constraints and assumptions

Type	Statement	Why it matters	How the team will respond
Priority	Protect sponsor-required work (required pages, secure sign in, unit tests) and security before optional features	Required work decides course and sponsor success	Cut Could items first, then Should; Must items and security are not cut
Constraint	Fixed dates: feature complete December 7 and final delivery December 15, 2026	There is no time after the semester	Feature freeze on December 7; the last week is for fixes, the scan, and the presentation
Constraint	Fabricated data only and a $0 budget, except about $1–10 of AI tokens	Limits hosting and tools	Local development, open-source tools, and an AI spending cap

Type	Statement	Why it matters	How the team will respond
Assumption	Each member can give about 10 hours per week	If wrong, planned scope slips	Checked at the weekly check-in; flexible team environment

2 Release strategy

A release plan shows the overall path to the final product. It stays less detailed than an individual sprint plan.

2.1 Major deliverables and milestones

ID	Deliverable or milestone	Evidence	Target date or sprint	Owner	Reviewer
M-01	Project charter approved	Pending	September 25, 2026 (updated since)	Cesar Giner	Instructor
M-02	Requirements baseline	Use cases, FRs, NFRs	October 2, 2026 (updated since)	Team	Instructor
M-03	Architecture Document	Architecture diagram and decisions	Nov 3 , 2026	Whole Team	Instructor
M-04	Midsemester Demo	Working demo and feedback notes	October 29, 2026 (done)	Team	Instructor and Commerce Bank mentors
M-05	Feature complete	Testable release candidate; ZAP report	December 7, 2026	RoyalDev team	Instructor
M-06	Unit tests and beta testing	Coverage report of at least 70%; tester feedback notes	December 1, 2026	Jacob Melookaran, Jake Hall	Team
M-07	Final delivery	Application, tests, documents, and presentation	December 15, 2026	RoyalDev team	Instructor and Commerce Bank mentors

2.2 Release and sprint roadmap

Each sprint has one goal and one demonstrable outcome. Sprints 0–3 are complete; Sprint 4 is in progress.

Sprint	Sprint goal	Planned demonstrable outcome	Main dependencies	Review evidence
Sprint 0 (Sep 4–17)	Align and prepare	Team roles, sponsor brief reviewed, app concept chosen	Sponsor brief and team access	Team agreement
Sprint 1 (Sep 18–Oct 1)	Prove the technical path	Stack and schema chosen; register and sign in work end to end	Charter, tech stack decision	Live demo and charter

Sprint	Sprint goal	Planned demonstrable outcome	Main dependencies	Review evidence
Sprint 2 (Oct 2–15)	Users can plan a month	Sessions, sign out, and monthly budgets with plans and allocations	Requirements baseline, sessions table	Live demo
Sprint 3 (Oct 16–29)	Plan vs. actual for the midterm	Transactions, budget progress, dashboard, Tailwind styling	Budgets API, Figma designs	Midterm demo (October 29)
Sprint 4 (Oct 30–Nov 12)	Close midterm feedback and complete the required pages	Profile and Settings page, post-demo fixes, test setup, documents v1.0	Midterm feedback, mentor answers	Demo, pytest run, documents in repo
Sprint 5 (Nov 13–26)	Add the AI assistant and logging	AI side-panel assistant, file logging, tests growing toward 70%	AI provider and API key, test setup	Demo and coverage report
Sprint 6 (Nov 27–Dec 7)	Reach feature complete, secure and tested	ZAP scan with 0 critical or high findings, 70% coverage, beta testing	All features merged	ZAP report, coverage report, tester notes
Sprint 7 (Dec 8–15)	Deliver the final product	Final fixes, optional deployment, final presentation	Feature-complete build	Final demo and presentation

3 Backlog and work breakdown

3.1 Prioritized backlog summary

Stories map to the use cases in the SRS and the epics on the Jira board (CP project).. Items US-01 to US-12 form the MVP; US-13 to US-16 are committed additions; the rest are optional.

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
US-01	Register an account (UC-AUTH-01)	Must	5 points	Sprint 1	Done
US-02	Sign in (UC-AUTH-02)	Must	3 points	Sprint 1	Done
US-03	Stay signed in with 12-hour sessions (UC-AUTH-03)	Must	5 points	Sprint 2	Done
US-04	Sign out (UC-AUTH-04)	Must	2 points	Sprint 2	Done
US-05	Create a monthly budget (UC-BUD-01)	Must	8 points	Sprint 2	Done
US-06	View budget progress, plan vs. actual (UC-BUD-02)	Must	5 points	Sprint 2	Done
US-07	Record a transaction (UC-FIN-01)	Must	5 points	Sprint 3	Done
US-08	View transaction history (UC-FIN-02)	Must	3 points	Sprint 3	Done
US-09	View the dashboard (UC-DASH-01)	Must	5 points	Sprint 3	Done

Backlog ID	Epic story or deliverable	Priority	Estimate	Target sprint	Status
US-10	Style all pages with Tailwind to match Figma	Must	5 points	Sprint 3	Done
US-11	Profile and Settings page (UC-PROF-01)	Must	5 points	Sprint 4	Active
US-12	Unit and API tests with at least 70% backend coverage	Must	8 points	Sprints 4–6	Active
US-13	AI budget assistant side panel (UC-AI-01)	Must	13 points	Sprint 5	Ready
US-14	File logging of errors and failed sign-ins (stretch goal)	Must	3 points	Sprint 5	Ready
US-15	OWASP ZAP scan and fixes (stretch goal)	Must	5 points	Sprint 6	Ready
US-16	Post-demo fixes: register redirect, block future dates, actual income, clearer projected savings, create-budget prompt	Should	5 points	Sprints 4–5	Active
US-17	Session inactivity timeout (30 minutes)	Should	3 points	Sprint 5	Proposed
US-18	Limit failed sign-in attempts	Should	3 points	Sprint 5	Proposed
US-19	Charts for spending by category	Should	5 points	Sprint 6	Proposed
US-20	Deploy the app for the final demo	Could	5 points	Sprint 7	Proposed
US-21	Notifications	Could	3 points	Not scheduled	Proposed
US-22	Project documents v1.0 (charter, SRS, use cases, architecture, plan)	Must	8 points	Sprints 1–4	Active
US-23	Final presentation and demo rehearsal	Must	5 points	Sprint 7	Ready

3.2 Work breakdown structure

Each leaf item below produces a reviewable output. Items 1.0 to 5.0 are complete except 1.4. Detailed work packages for the remaining work are in *section 3.3 'Work Package Details'*

1.0 Project management and documents: 1.1 Charter; 1.2 Requirements (SRS) and use cases; 1.3 Architecture document; 1.4 Project plan; 1.5 Jira backlog and sprint planning.

2.0 Foundation: 2.1 Tech stack decision; 2.2 GitHub repository and folders; 2.3 Database schema and seed categories; 2.4 Local setup guide (README).

3.0 Identity: 3.1 Users and sessions tables; 3.2 Register API and page; 3.3 Sign-in API and page; 3.4 Sessions (12 hours) and sign out.

4.0 Budgeting: 4.1 Budgets API with plans and allocations; 4.2 Budget page (create and progress); 4.3 Shared budget math (budgetMath.js).

5.0 Transactions and dashboard: 5.1 Transactions API; 5.2 Transactions page; 5.3 Dashboard; 5.4 Tailwind styling.

6.0 Profile and settings: 6.1 Profile API (view, update, change password); 6.2 Profile and Settings page.

7.0 AI assistant: 7.1 Choose provider and set a spending cap; 7.2 AI endpoint with budget summary; 7.3 Side-panel chat UI.

8.0 Quality and security: 8.1 Test setup (pytest, test database); 8.2 Unit and API tests to 70%; 8.3 File logging; 8.4 ZAP scan and fixes; 8.5 Beta testing.

9.0 Release: 9.1 Post-demo fixes; 9.2 Optional deployment; 9.3 Final presentation and rehearsal.

3.3 Work package details

WBS ID	Work package	Owner	Output and completion test	Effort	Duration	Predecessor
6.1	Profile API: view, update name and email, change password	Cesar Giner	Changes saved; password change needs the current password; tests pass	6 hours	4 days	3.4
6.2	Profile and Settings page	Jake Hall	Page shows and edits the profile with clear errors	6 hours	4 days	6.1
7.1	Choose AI provider and set a spending cap	Cesar Giner	Provider chosen (Q-001); key only in the backend .env; cap set	2 hours	3 days	None
7.2	AI endpoint with budget summary	Cesar Giner	Answers questions; shows "unavailable" on failure; per-user message limit	10 hours	1 week	7.1, 4.1, 5.1
7.3	AI side-panel chat UI	Jake Hall	Freeform panel opens from every page and shows answers	8 hours	1 week	7.2
8.1	Test setup: pytest, FastAPI TestClient, test database	Jacob Melookaran	pytest runs green on a fresh clone	4 hours	3 days	None
8.2	Unit and API tests to 70% coverage	Jacob Melookaran, Cesar Giner	Coverage report at least 70%; all tests pass	16 hours	3 weeks	8.1
8.3	File logging	Joshua Webber	Errors and failed sign-ins logged to a file with no passwords or tokens	4 hours	3 days	None
8.4	ZAP scan and fixes	Joshua Webber	Final report shows 0 critical or high findings	8 hours	2 weeks	8.3, 6.2, 7.3
8.5	Beta testing with testers	Jake Hall	Testers complete register → budget → transactions → plan vs. actual; notes recorded	6 hours	1 week	6.2
9.3	Final presentation and rehearsal	Whole team	Slides done; demo rehearsed twice with fabricated data	10 hours	1 week	8.4, 8.5

4 Estimation and capacity

4.1 Estimation method and assumptions

Stories are estimated in story points (1, 2, 3, 5, 8, 13) by the whole team at sprint planning; remaining work packages are estimated in person-hours. Estimates include coding, review, testing, and documentation. They

are revised at each sprint planning using the points actually completed in earlier sprints (about 15–20 points per sprint so far).

Assumptions: most members are new to full-stack development, so new tasks (testing, the AI assistant) get extra time; development is local; these are estimates for a semester project and are adjusted as the team learns.

4.2 Effort and duration

Effort means labor time. Duration means calendar time. A task may require two hours of effort but three days of duration while the team waits for a review or external decision.

Task	Effort estimate	Duration estimate	Reason for difference	Confidence
AI provider setup	2 person-hours	3 days	Account, billing, and spending cap setup(no exp)	Medium
ZAP scan and fixes	8 person-hours	2 weeks	Baseline scan, fixes, and a final re-scan(no exp)	Medium
Beta testing	6 person-hours	1 week	Depends on testers' availability	Low
Final presentation	10 person-hours	1 day	Rehearsals need the whole team together for a couple hours, one day	High

4.3 Team capacity

Capacity assumes about 10 hours per week per person, with a focus factor for classes, jobs, and meetings. Thanksgiving week (November 23–29) reduces Sprints 5 and 6. Hours can change from week to week; the team updates them at sprint planning.

Team member	Role	Hours available per week	Planned absence	Focus factor	Usable sprint capacity	Backup
Cesar Giner	Project lead; front-end and API developer	12	Thanksgiving week	75%	18 hours	Jacob Melookaran
Jacob Melookaran	Database lead	10	Thanksgiving week	70%	14 hours	Cesar Giner
Jake Hall	UI/UX design	10	Thanksgiving week	70%	14 hours	Cesar Giner
Joshua Webber	Security architect; documentation	10	Thanksgiving week	70%	14 hours	Team consensus

5 Schedule and dependencies

5.1 Task dependencies

From task	To task	Dependency type	Explanation	Risk if delayed
Database schema	All APIs	Finish to start	Every endpoint maps to agreed tables	All feature work slips
Sessions (sign in)	Budget, transaction, and profile APIs	Finish to start	Protected endpoints need get_current_user	Every protected feature slips
Budgets and transactions APIs	Dashboard and AI budget summary	Finish to start	Both read budget and transaction data	Dashboard and AI slip
Profile API	Profile and Settings page	Start to start	UI work can begin once the endpoint shapes are agreed	Required page late
AI provider choice	AI endpoint	Finish to start	The endpoint needs a key and an API format	AI assistant slips to Sprint 6
Test setup	Coverage work	Finish to start	Tests need pytest and a test database	70% coverage missed
Feature development	Final ZAP scan	Finish to finish	The final scan must run after the last feature is merged	Unscanned code at release

5.2 Schedule

Figure 1 shows the sprint calendar, the main work streams, and the milestones. The red line marks the plan date (November 9). Day-to-day tasks are tracked on the Jira board.

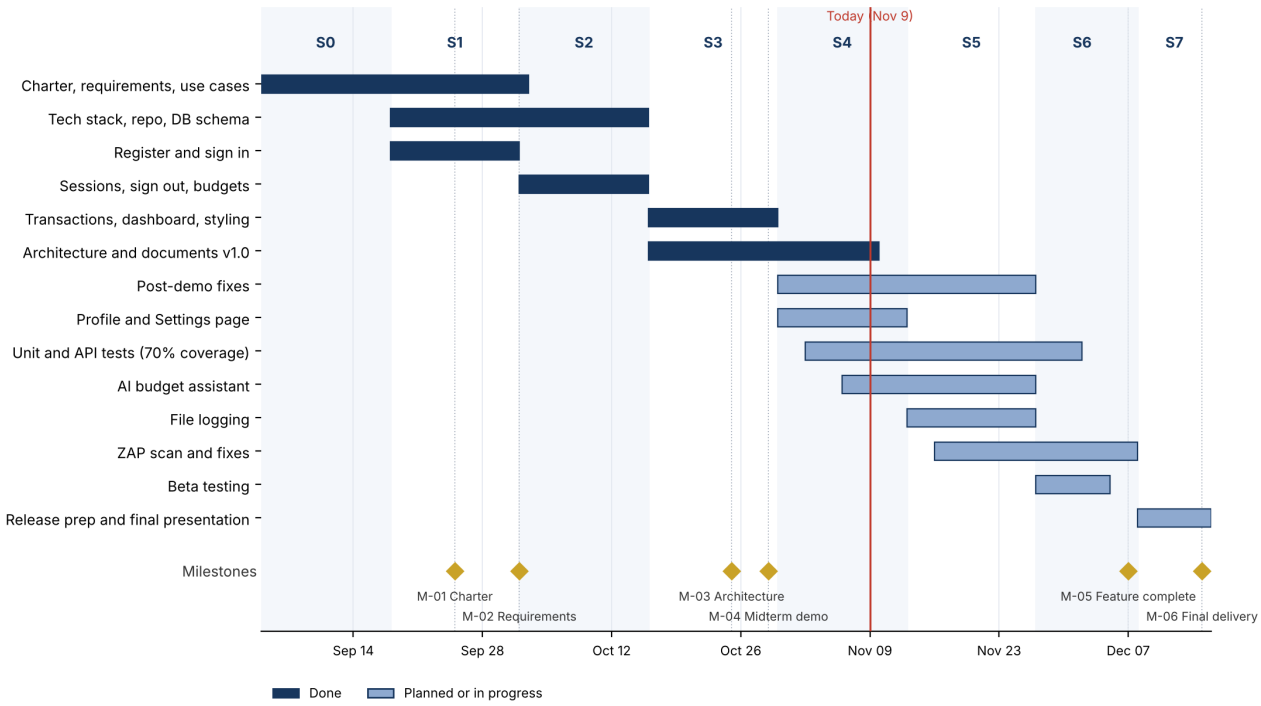


Figure 1. Schedule (v1.0, November 9, 2026). Dark bars are done; light bars are planned or in progress.

5.3 Critical work and float

These paths control the remaining milestones. Slack is the time between the planned finish and the milestone.

Critical task or path	Deadline controlled	Available slack	Early warning	Response
AI provider → AI endpoint → chat UI	Feature complete (December 7)	About 1 week	Endpoint not answering by November 26	Ship a simple question-and-answer version; keep the "unavailable" fallback
Test setup → tests to 70% coverage	Unit tests (December 1)	About 3 days	Coverage under 40% by November 20	Pair on tests; cover routes and budget rules first
Feature freeze → ZAP scan → fixes → re-scan	Feature complete and final delivery	About 3 days	A high finding after December 3	Run a baseline scan early in Sprint 5; fix highs first
Rehearsal → final presentation	Final delivery (December 15)	About 2 days	No full rehearsal by December 12	Freeze demo data; record a backup demo video

5.4 Resource balancing

Cesar Giner has built most of the front-end and API and also coordinates the documents, so he is the main bottleneck. For the remaining sprints, testing moves to Jacob Melookaran, logging and the ZAP scan to Joshua Webber, and the Profile page UI, chat panel UI, and beta testing to Jake Hall.

Cesar focuses on the AI endpoint and the Profile API, pairs on unfamiliar work, and reviews changes before they are merged. If someone falls behind, the work is rebalanced at the weekly check-in.

6 Sprint operating plan

6.1 Sprint cadence

Event	Purpose	Participants	Timing and timebox	Output
Sprint planning	Choose a sprint goal and work that fits capacity	Whole team	First day of the sprint (Friday), 60 minutes	Sprint goal and sprint backlog in Jira
Weekly check-in	Share progress and blockers	Whole team	Weekly, in person at the UMKC library, 30 minutes; short updates in Discord between	Updated Jira board and follow-up actions
Review or demo	Show working software and get feedback	Team, mentors at check-ins, instructor	Last day of the sprint (Thursday), 30 minutes	Feedback and acceptance decisions
Retrospective	Improve the team's way of working	Student team	After the review, 20 minutes	One or two improvement actions

6.2 Sprint plan

Sprint field	Team entry
Sprint number and dates	Sprint 4: October 30 – November 12, 2026
Sprint goal	Close the midterm feedback, finish the v1.0 documents, and deliver the Profile and Settings page with a working test setup.
Committed stories or deliverables	US-11 Profile and Settings page; US-12 test setup (WBS 8.1); US-16 post-demo fixes (first part); US-22 documents v1.0
Capacity and known absences	About 60 person-hours; no planned absences
Main dependencies	Profile API before the Profile page; mentor answers on the stretch goals
Review evidence	Profile page demo, a passing pytest run, and documents
Plan adjustment rule	Protect the Profile page and test setup; move remaining post-demo fixes to Sprint 5 first.

6.3 Definition of Ready and Definition of Done

Definition	Required conditions
Ready	The story is linked to a use case or requirement; acceptance criteria are clear; the API shape and data are agreed; dependencies are done or planned; the story is estimated and fits in one sprint.
Done	Acceptance criteria pass; the code runs locally without errors; relevant tests pass; the feature was tested end to end in the browser; the code was reviewed (pull request for features) and merged to main; update docs

7 Roles and responsibilities

Role or person	Main responsibilities	Decision authority	Backup	Expected evidence
Cesar Giner, project lead and front-end/API developer	Maintains the plan and backlog, runs sprint planning and check-ins, builds pages and endpoints, merges to main	Routine plan updates, front-end architecture, integration	Jacob Melookaran	Current plan, README, working increments
Jacob Melookaran, back-end lead	Database schema, test setup and API tests;	Database schema and back-end design	Cesar Giner	schema.sql, test and coverage reports
Jake Hall, UI/UX design	Figma designs, page layouts, accessibility checks, beta testing	UI/UX design choices	Cesar Giner, Joshua	Figma files, usability notes
Joshua Webber, security architect and documentation	Security standards, file logging, ZAP scan, document reviews	Security standards and documentation	Team consensus	ZAP report, reviewed documents

Role or person	Main responsibilities	Decision authority	Backup	Expected evidence
Commerce Bank mentors	Answer requirement questions and review demos	Interpretation of sponsor requirements	Course instructor	Check-in feedback

8 Communication plan

Decisions are made in team meetings and recorded in Jira, the README, or the project documents,

Information	Creator or source	Audience	Method or location	Frequency or trigger	Owner
Sprint status	Jira board and team	Instructor and Commerce Bank mentors	Short update plus demo	End of each sprint and at mentor check-ins	Jacob
Technical decision	Team discussion	Team	ADR in the architecture document; Jira Sprint task	When a significant decision is made	Team
Day-to-day coordination	Team	Team	Discord	As needed; replies within 24 hours	All members
Weekly progress	Team	Team	In-person check-in at the UMKC library	Weekly	Team
Requirement questions	Team	Commerce Bank mentors	Email or mentor check-in	When a question comes up	Team
Task status	Task owner	Team	Jira board (CP project)	Whenever a task changes status	Task owner

8.1 Team communication rules

Discord is the primary channel, and members reply within 24 hours. The team meets in person weekly at the UMKC library; members come prepared and keep discussion respectful and on topic. Decisions are written down in Jira or the documents. Conflicts are first discussed within the team; if they remain unresolved, the team brings them to the course instructor.

9 Quality plan

Quality activity	When it occurs	Owner	Entry condition	Completion evidence
Code review	Before merging a feature branch	Another team member	Runs locally without errors	Approved pull request or review note
Unit testing(planned)	Should occur during implementation	Team	Testable behavior identified	Passing pytest run and coverage report
Security scan	Baseline in Sprint 5; final in Sprint 6	Joshua Webber	App runs locally with fabricated data	ZAP report with 0 critical or high findings

Quality activity	When it occurs	Owner	Entry condition	Completion evidence
Beta testing	Sprint 6	Jake Hall	Main flows feature complete	Tester notes (SC-02)
Document review	Each new document version	Joshua Webber	Draft complete	Revision history entry and PDF in docs/

9.1 Defect handling

Severity levels: Critical (data exposure, security finding rated high or critical, or the app does not start); High (a core flow such as sign in, budgets, or transactions is broken); Medium (a feature is wrong but has a workaround); Low (cosmetic or wording issues).

Defects are noted by individuals which is then communicated to the team, critical/high defects are handled promptly, else are listed and issue to resolve at a later date.

10 Risks and issues

10.1 Risk register

Risk ID	Possible problem	Probability	Impact	Trigger	Mitigation and contingency	Owner
R-01	Uneven workload; one person carries most of the coding	Medium	High	One person does most of a sprint's work	Rebalance work, pair on unfamiliar tasks, enforce collaboration	Team
R-02	Requirements misunderstood (stretch goals, required pages)	Low	Medium	Mentor feedback contradicts the plan	Confirm open questions with the mentors by email	Team
R-03	Security issues found late	Medium	High	High finding after December 3	Baseline ZAP scan in Sprint 5; fix highs first	Joshua Webber
R-04	Miscommunication within the team	Medium	High	Missed replies or duplicate work	24-hour replies on Discord; weekly check-in; tasks assigned in Jira	Team
R-05	Planned scope does not fit the semester	Medium	High	Remaining points exceed capacity at sprint planning	Cut Could, then Should items; protect Must items	Team
R-06	AI provider cost or outage	Medium	Medium	Spending near the cap or API errors	Spending cap, per-user message limit, fallback message	Cesar Giner
R-07	Final demo fails on presentation day	Medium	High	Errors during rehearsal	Rehearse, reset demo data, record a backup video	Team

10.2 Issue log

Issue ID	Current problem	Effect	Action	Owner	Due date	Status

Issue ID	Current problem	Effect	Action	Owner	Due date	Status
I-01	Jira board is out of date; many finished tasks still show To Do, Update Jira	Progress is hard to see	Update all issues and create Sprints 4–7	Cesar Giner, Jacob Melookaran	November 12, 2026	Open
I-02	No automated tests yet	Sponsor requirement not yet met	Set up pytest and a test database (WBS 8.1)	Jacob Melookaran	November 12, 2026	Open
ID	-	-	-	-	-	-

11 Change and configuration control

11.1 Scope and plan change process

Step	Team procedure
1 Request	Record the requested change, reason, requester, and date
2 Analyze	Estimate value, effort, schedule, risk, quality, and document effects
3 Decide	Routine changes within a story: the story owner. Changes to scope, schedule, or architecture: the whole team by consensus. Unresolved disagreements go to the instructor. Changes to sponsor requirements are confirmed with the Commerce Bank mentors.
4 Update	Revise backlog, requirements, plan, architecture, tests, and decision log as needed
5 Communicate	Tell affected stakeholders what changed and why

11.2 Configuration management

The GitHub repository (Cesar-KC/RoyalDevCapstoneProject) has one folder per part: front-end/, back-end/, database/, and docs/. The main branch must always run. Larger features are built on a feature branch (feature/<name>) and merged into main through a pull request after local testing. Small fixes and document updates may be committed directly to main.

Documents are stored as PDFs in docs/, with the version in the file name (for example, v1.0); older versions are kept and marked (old). Releases are tagged in Git: v0.5-midterm for the midterm increment and v1.0 for final delivery. The repository is public, only team members can push, and secrets (.env files) are never committed.

12 Progress tracking

Measure	How calculated or judged	Source	Frequency	Decision it supports
Sprint goal result	Met, partially met, or not met, with explanation	Sprint review	Each sprint	Adjust scope or process
Velocity	Story points moved to Done in the sprint	Jira board	Each sprint	How much to commit next sprint

Measure	How calculated or judged	Source	Frequency	Decision it supports
Forecast	Remaining points compared with capacity before December 7	Backlog and capacity	Each sprint	Move, split, or cut work
Test and defect status	Tests passing, coverage percentage, and open defects by severity	pytest coverage report and Jira	Weekly from Sprint 4	Release readiness

12.1 Status report format

Current Short Report:

Completed Work: Update documentation, Analyze Feedback, Profile API started.

Next: Profile page UI and test setup

Blockers: UI decisions

Risk changes: No tess

Quality: 0 tests, 0% coverage, 2 open bugs (1 medium, 1 low).

Forecast: on track for feature complete on Dec 7, as long as the planned AI work is on schedule(sprint 5)

Decisions needed: pick the AI provider

13 Tools and environments

Purpose	Tool or environment	Location	Access owner	Use and backup notes
Source control	GitHub	github.com/Cesar-KC/RoyalDevCapstoneProject	Cesar Giner	main always runs
Project board	Jira (CP project)	umkc-team-hlh86r9q.atlassian.net, project CP	Jacob Melookaran	Statuses: To Do, In Progress, Done; the owner moves each issue
Design	Figma	Team Figma project	Jake Hall	Designs are the reference for page styling
Communication	Discord, In-Person	Team server	Team	Replies within 24 hours
Development	Local: VS Code, Python 3.13 (venv), Node.js 24, MySQL 9	README setup section	Each developer	.env created from .env.example and never committed
Test or demo	Local laptop; hosting planned (not decided)	http://localhost:5173	Cesar Giner	Reset with schema.sql and fabricated demo data before each demo

14 Review and submission

14.1 Review decisions

Reviewer	Role	Decision	Conditions or comments	Date
Commerce Bank mentors	Project sponsor	Pending		
Shah, Syed Jawad Hussain	Instructor	Pending		
Cesar Giner	Student project lead	Approved	Feedback welcomed	11/08/2026

14.2 Final submission checklist

☑ The plan agrees with the charter, requirements, architecture, and course calendar.

☑ Mandatory work and the MVP appear before optional or stretch work.

☑ Every sprint has a clear goal and demonstrable outcome.

☑ The WBS reaches schedulable work packages with owners and completion tests.

☑ Effort, duration, dependencies, capacity, and planned absences are realistic.

☑ Testing, integration, documentation, reviews, and defect work appear in the plan.

☑ Communication, risk, change, configuration, and progress practices are clear.

☑ The plan records what changed after the latest sprint review or retrospective.

☑ All bracketed prompts and unused example rows have been removed.

☐ The version, date, author list, and approval decisions are current. (Approval decisions pending.)